

"به نام خدا"

گزارش تمرین کامپیوتری اول

سیگنال ها و سیستم ها

دکتر اخوان

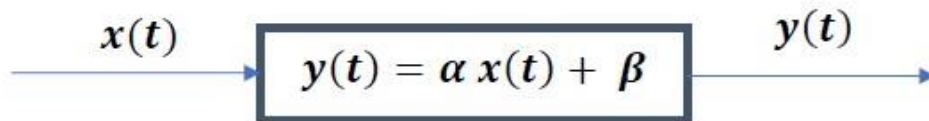
نازین نیکخواه بهرامی ۱۱۰۱۲۵۳۰

تمرین ۱:

این قسمت طبق خود فایل گزارشی نداشت و کد های آن درون فایل قرار دارد. یک قسمت به بررسی انواع متغیرها، ماتریس و ویژگی های آن ، بردار و نحوه نمایش آنها و ... پرداخته شده است و در یک بخش دیگر آن نحوه رسم نمودار.

تمرین ۲:

سیستم زیر را در نظر بگیرید که یک رابطه ی خطی را نشان می دهد.



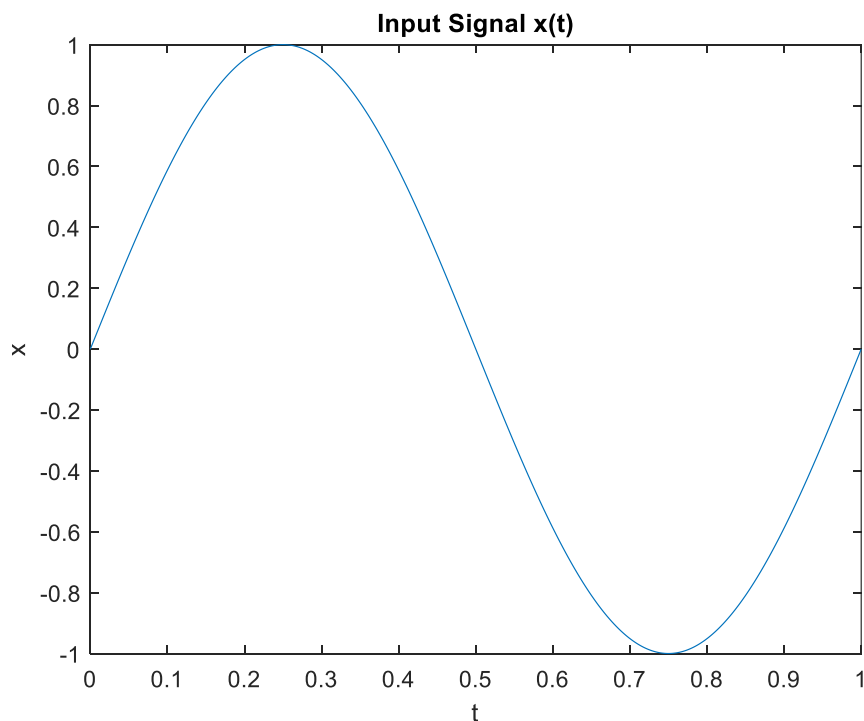
```
clc;  
clearvars;  
load('p2.mat');
```

برای لود کردن و آوردن دیتا این فایل ابتدا آن را در متلب گذاشته و بعد به کمک دستور بالا متغیر ها را میخوانیم.

تمرین ۲-۱)

برای رسم نمودار از روش سوال اول کمک میگیریم و متغیر ها را بر حسب هم نمایش میدهیم. محور طول و عرض را نیز بر اساس متغیر های هر قسمت نام گذاری میکنیم.

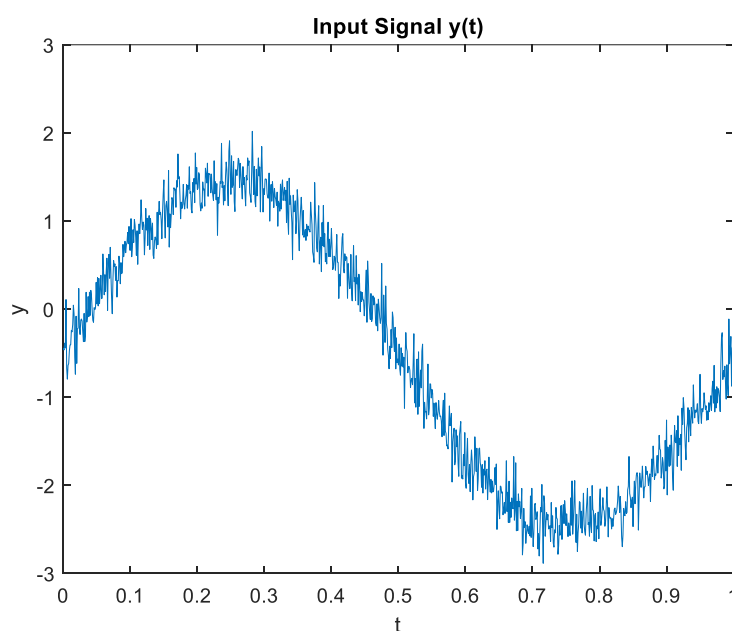
```
figure;  
plot(t,x);  
xlabel('t');  
ylabel('x');  
title('Input Signal x(t)');
```



تمرین ۲-۲)

برای نزدیک شدن به حالت واقعی باید از نویز استفاده کرد:

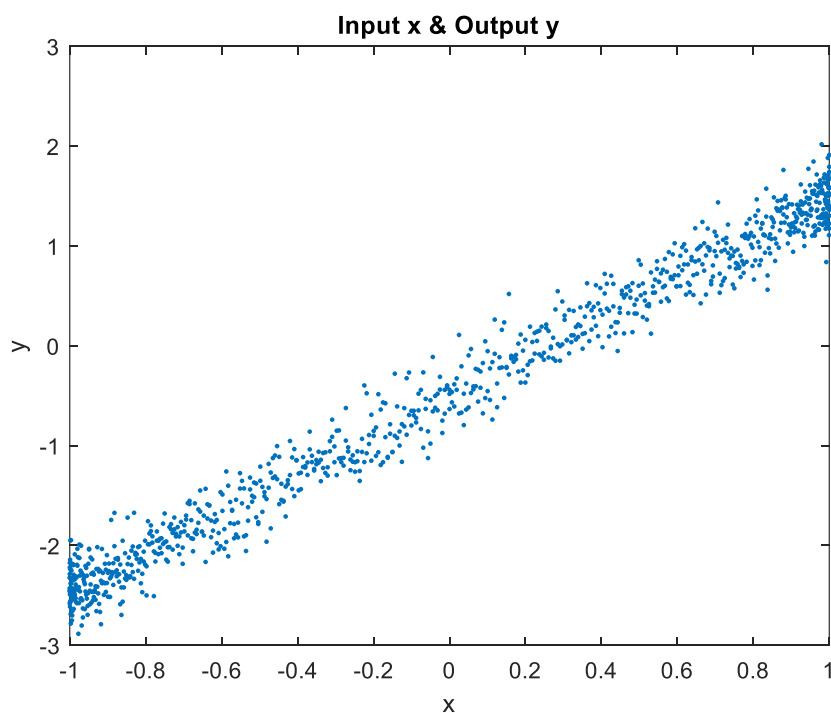
```
figure;  
plot(t,y);  
xlabel('t');  
ylabel('y');  
title('Input Signal y(t)');
```



تمرین ۲-۳)

برای اینکه رابطه دو متغیر را بهتر ببینیم از نقاط استفاده میکنیم. این رابطه همانطور که در شکل معلوم است حدوداً یک خط است و شیب آن آلفا و عرض از مبدا آن بتا را میدهد.

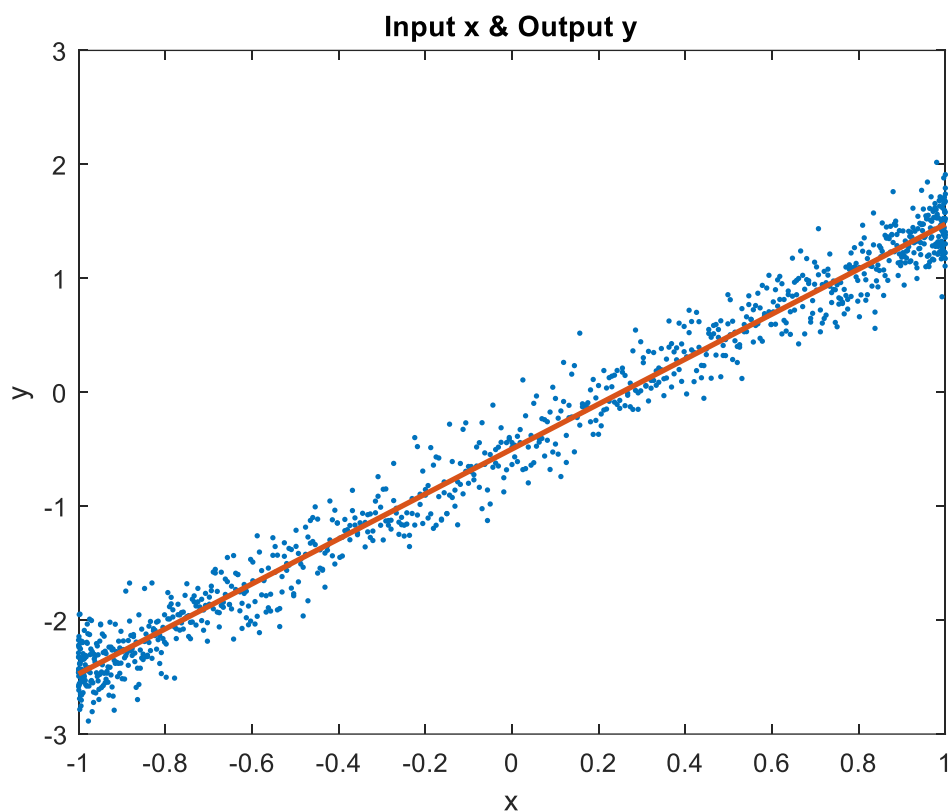
```
figure;  
plot(x,y,'.');  
xlabel('x');  
ylabel('y');  
title('Input x & Output y');
```



تمرین ۲-۴)

```
[alpha, beta] = p2_4(x, y)
figure;
plot(x,y,'.');
xlabel('x');
ylabel('y');
hold on;
time=-1:0.001:1;
plot(time,alpha*time+beta,LineWidth=2);
title('Input x & Output y');
```

با استفاده از تابع نوشته شده در این قسمت خروجی بصورت:



Command Window

alpha =

1.9736

beta =

-0.4983

حالا به بررسی تابع نوشته شده میپردازیم.

```
function [alpha, beta] = p2_4(x, y)
    x = x(:);
    y = y(:);
    N = length(x);
    A = [x ones(N,1)];
    theta = (A' * A) \ (A' * y);
    alpha = theta(1);
    beta = theta(2);
end
```

توجه کنیم که میتوانستیم از رگرسیون نیز استفاده کنیم یعنی از تابع هزینه داده شده درواقع بر حسب متغیرهای خروجی مدنظر که اینجا آلفا و بتا است مشتق بگیریم و مساوی صفر میگذاریم تا اکسترم پیدا شود و این را در یک حلقه قرار میدهیم.

اما از یک روش دیگر برای بهینه سازی نیز میتوان استفاده کرد که و معادله ماتریسی در متلب نوشت که ذخیره سازی آن(همانطور که در کلاس نیز بررسی شد) برای این برنامه حداقل مربعات سیستم خطی بهتر است. حالا برای شروع تابع باید دو ورودی و دو خروجی داشته باشد. حالا در اینجا یک الگوریتمی برای آن میتوان نوشت.

$$y = \alpha x + \beta \Rightarrow \begin{cases} y_1 = \alpha x_1 + \beta \\ y_2 = \alpha x_2 + \beta \\ y_3 = \alpha x_3 + \beta \\ \vdots \\ y_n = \alpha x_n + \beta \end{cases} \Rightarrow \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \alpha \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \beta$$

خط

α : شیب (مجهول)

β : عرض از مبدأ (مجهول)

x, y : نقاط (معلوم)

$$\Rightarrow \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \alpha \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \beta \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} \alpha \\ \beta \end{bmatrix}}_{\theta} \underbrace{\begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix}}_A$$

ستونی ستونی ستونی

↓

θ A

تواند تابع ما محروم
در دس x, y را استخوان کند

$\left\{ \begin{array}{l} \text{ones}(N, 1) \\ N = \text{length}(x) \rightarrow \end{array} \right.$ به تعداد n ها است

$\Rightarrow Y = \theta A \Rightarrow$ * ما در این جا به دنبال θ هستیم (روش حداقل مربعات برای رسم خط)

$\min (Y - \theta A)^2$

• بنابراین باید به نحوی مشتق گرفت در این جا از تراشه A استفاده میکنیم و بعد ماتریس را با ضرب در واریان حل میکنیم: در این جا θA و $A \theta$ میان اند (مفروضات خط را می دهند)

$$Y = A \theta \rightarrow A^T Y = (A^T A) \theta \rightarrow (A^T A)^{-1} A^T Y = \theta \Rightarrow \alpha, \beta \text{ بدست می آیند}$$

"معادله نرمال"

چک کردن تابع قسمت تمرین ۲-۴)

برای چک کردن تابع باید یک سری داده بدهیم و ببینیم در دو حالت با نویز کم قدرت و بدون نویز، خروجی درست میدهد یا خیر. داده های مصنوعی به این شکل است:

```
N = 100;  
x = linspace(-10, 10, N)';  
actual_alpha = 2.5;  
actual_beta = -1.0;  
y = actual_alpha * x + actual_beta;
```

حالا بعد از اینکه بصورت دستی داده ها و پارامتر ها را وارد کردیم از تابع برای تخمین استفاده میکنیم:

```
[alpha_no_noise, beta_no_noise] = p2_4(x, y);
```

در مرحله بعد یک نویز کم به تابع میدهیم و آن را بصورت مجموعه داده رندوم گوسی از بازه مدنظر (میانگین ۰ و واریانس ۱) انتخاب میکنیم (این روش در کلاس نیز مطرح شد):

```
noise_level = 1.0;  
y_noisy = y + noise_level * randn(size(y));
```

دوباره آن را در تابع قرار میدهیم:

```
[alpha_with_noise, beta_with_noise] = p2_4(x, y_noisy);
```

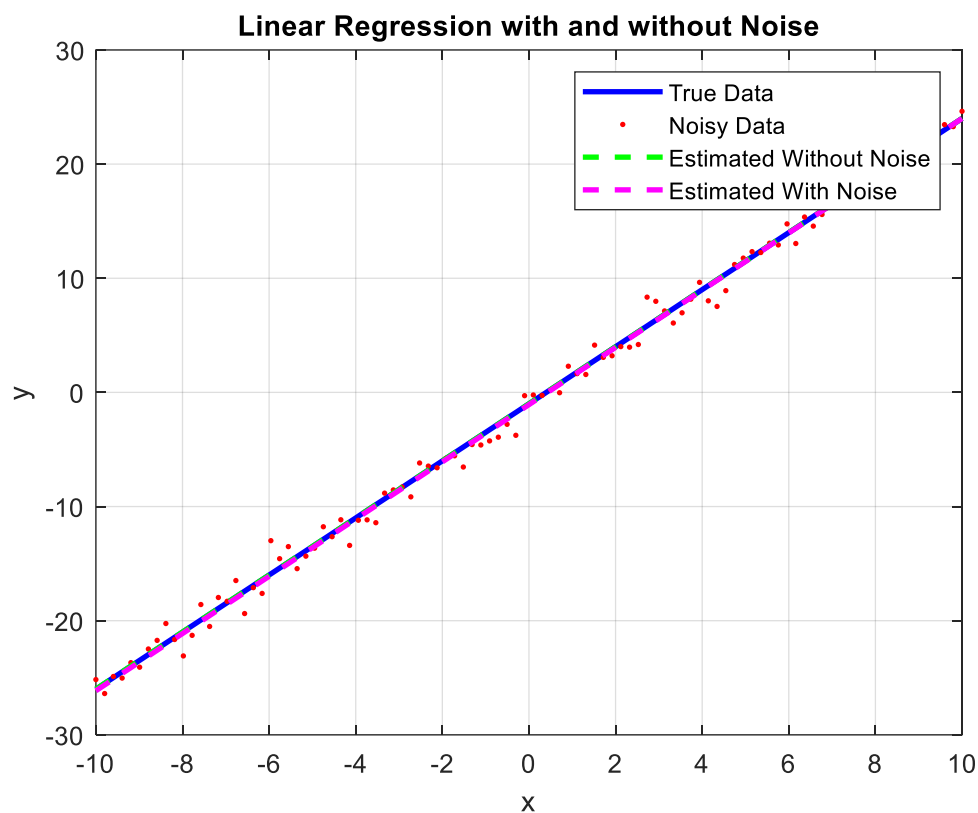
ابتدا همه موارد را خروجی میگیریم و مقایسه میکنیم: مثلاً یک مورد خروجی عددی

```
fprintf('Actual Alpha: %.2f, Actual Beta: %.2f\n', actual_alpha, actual_beta);  
fprintf('Estimated Alpha (No Noise): %.2f, Estimated Beta (No Noise): %.2f\n', alpha_no_noise,  
beta_no_noise);  
fprintf('Estimated Alpha (With Noise): %.2f, Estimated Beta (With Noise): %.2f\n', alpha_with_noise,  
beta_with_noise);
```

```
Command Window  
  
Actual Alpha: 2.50, Actual Beta: -1.00  
Estimated Alpha (No Noise): 2.50, Estimated Beta (No Noise): -1.00  
Estimated Alpha (With Noise): 2.51, Estimated Beta (With Noise): -1.07  
fx >>
```

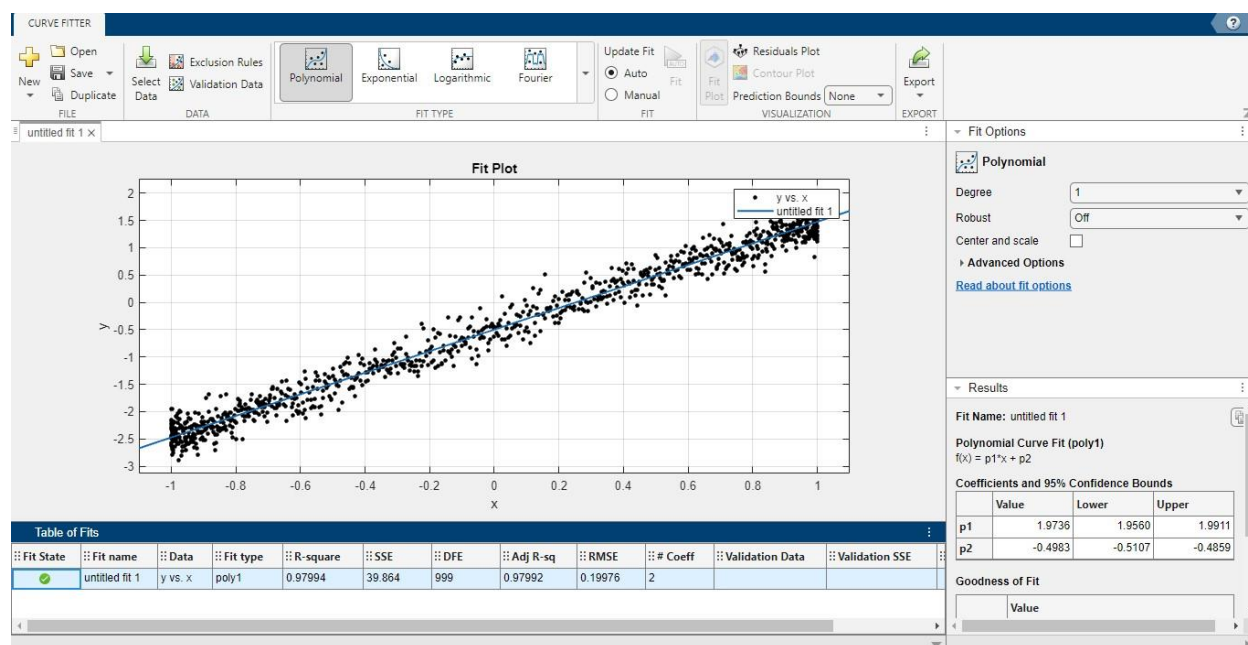
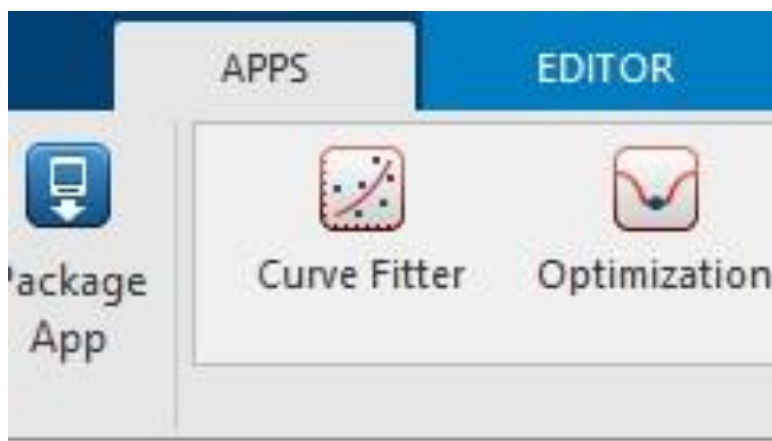

همانطور که از خروجی ها معلوم است، تابع درست کار میکند و در حالت نویز نیز دارد خروجی نزدیکی میدهد (بعلت رندوم بودن نویز خروجی هر سری ممکنه است فرق کند) اما میتوان بصورت نموداری نیز بررسی کرد تا ببینیم آیا داده ها روی هم تطبیق دارند یا خیر.

مثلا یک مورد خروجی پلات:



بنابراین تابع درست کار میکند.

تمرین ۲-۵)



همانطور که مشخص است، خروجی ها بسیار تطبیق دارند.

تمرین ۳:

می خواهیم توسط یک رادار، فاصله ی یک جسم از رادار را بیابیم.



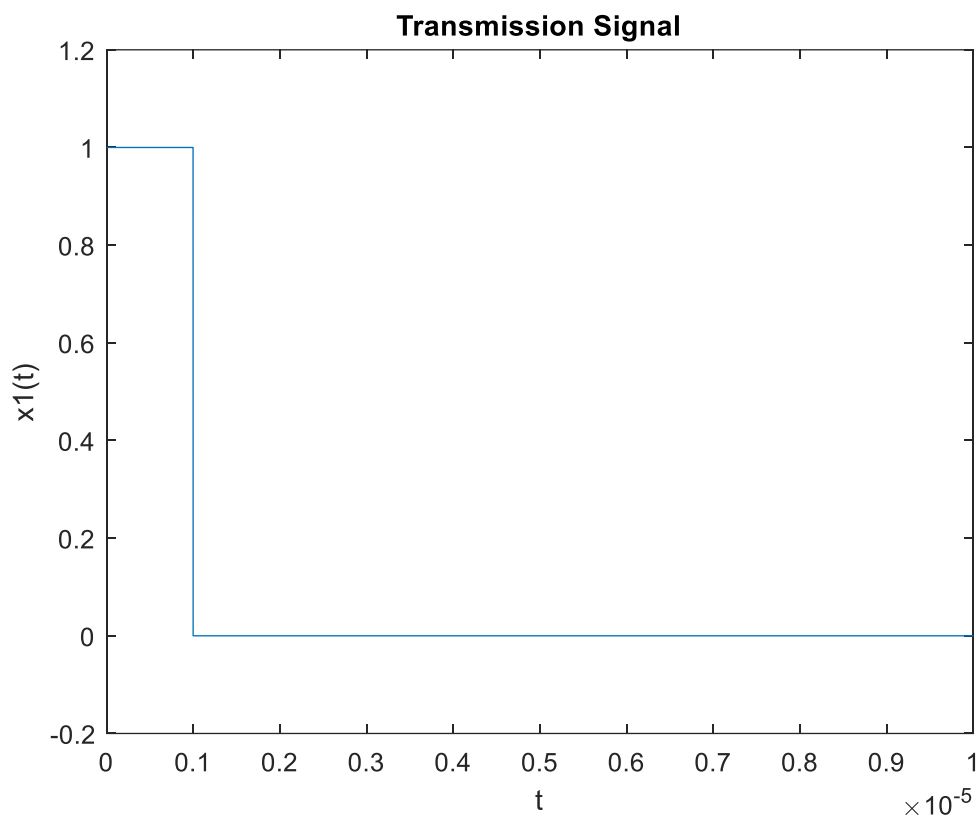
تمرین ۳-۱)

سیگنال ارسالی رادار به صورت زیر است:



$$ts=1e-9; T=1e-5; tau=1e-6;$$

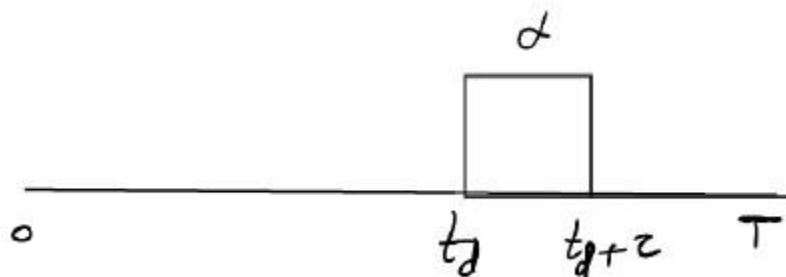
```
clc;
clearvars;
close all;
ts= 1e-9;
T= 1e-5;
tau= 1e-6;
t= 0:ts:T;
x=zeros(size(t));
انگار یک فضای وکتوری با آرایه ای به طول زمان درست کرده ایم
اندازه ثابت ساخته شده (این جا ممکنه است وارنینگ اینتجر بدهد);
x(1:(tau/ts))=1;
figure;
plot(t,x);
xlim([0,T]);
ylim([-0.2 1.2]);
این فاصله ها برای مرتب شدن پشت سیگنال و از بین نرفتن قسمتی از آن است;
xlabel('t');
ylabel('x1(t)');
title('Transmission Signal');
```



بنابراین سیگنال ورودی با اطلاعات داده شده رسم شد.

علت وارنینگ: چون با اندیس (اینتر) کار میکنیم و تقسیم میکنیم چون ممکن است جوابش عدد صحیح نباشد، خطای تصحیح میدهد.

تمرین ۲-۳)



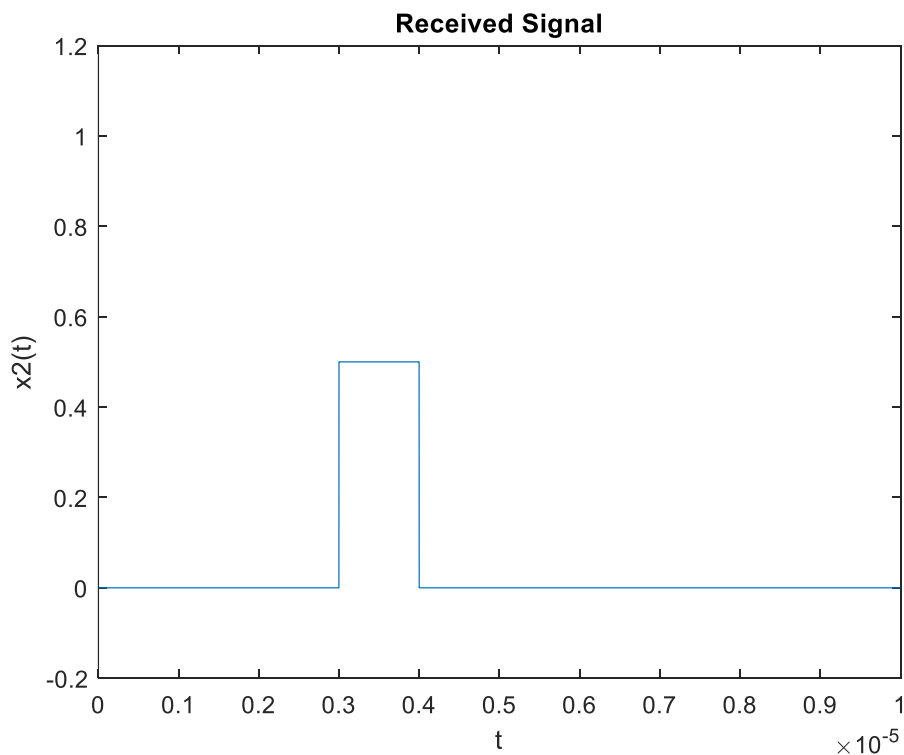
که در شکل بالا $t_d = \frac{2R}{c}$ است و مقدار $\alpha < 1$ اهمیتی ندارد. در این رابطه C سرعت نور با واحد متر بر ثانیه و R فاصله ی رادار تا جسم با واحد متر است.

```
R=450;  
c=3e8;  
td=(2*R)/c;  
Received_Signal=zeros(size(t));  
alpha=0.5;  
Received_Signal((td/ts):((td+tau)/ts))=alpha;  
figure;  
plot(t,Received_Signal);  
xlim([0,T]);  
ylim([-0.2 1.2]);  
xlabel('t');  
ylabel('x2(t)');  
title('Received Signal');
```

دوباره با همین روش یک سیگنال با مقدار صفر تولید میکنیم که طول آن اندازه زمان است و بعد

با مقادیر پر میشود با این تفاوت که اینجا ضریب تضعیف نیز داریم. همان مقادیرهای قبلی برای

محورهای نمودار استفاده میکنیم. خروجی به این صورت است:



تمرین ۳-۳

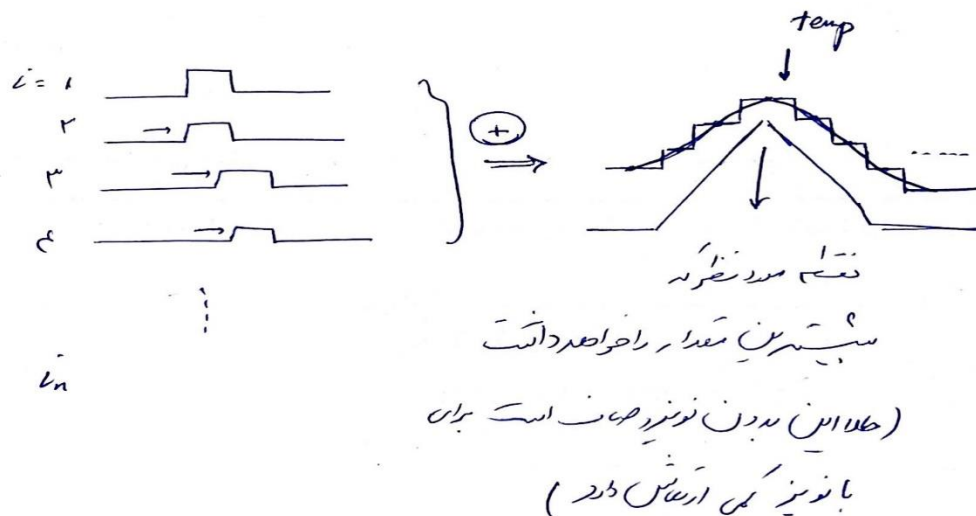
به کمک روش کوررلیشن یا تمپلیت مچینگ میخواهیم دو نمودار را انقدر جمع ببریم و جلو ببریم تا جایی که بتوان ماکسیمم را پیدا کرد.

```
function ro = p3_corfunc(t, tau, ts, Transmission_Signal)
    ro = zeros(size(t));
    for i=1:length(t)-int64(tau/ts)
        temp_signal = zeros(size(t));
        temp_signal(i:i+(tau/ts)-1) = 1;
        ro(i) = sum(temp_signal .* Transmission_Signal);
    end
end
```

این تابع ۴ ورودی شامل زمان، تاو، فاصله دو نمونه زمانی و سیگنال دریافتی را میگرد و به ما یک خروجی "رو" را میدهد. طبق آنچه در آمار و احتمال داشتیم برای کوررلیشن گیری داریم:

$$\text{corr}(x, y) = \frac{n \sum_{i=1}^n x_i y_i - \sum_{i=1}^n x_i \sum_{i=1}^n y_i}{\sqrt{(n \sum_{i=1}^n x_i^2 - (\sum_{i=1}^n x_i)^2)(n \sum_{i=1}^n y_i^2 - (\sum_{i=1}^n y_i)^2)}}$$

که این فرمول که بررسی میکند برای دو بعد است اما اینجا ما فضای ذخیره سازی را یک بعدی در نظر میگیریم که ابتدا ۰ است و بعد داخل آن مجموع ها حفظ میشود. بعلت وارنینگ گفته شده بالاتر و طولانی شدن پروسه، از اینتجر استفاده میکنیم. این تابع در یک حلقه با بازه زمانی مدنظر تا انتهای زمان جلو رفته و سیگنال هارا ۱ میکند (مربع میکشد، این همان روش ابتدای تمرین است) و رسم میکند حالا برای کوررلیشن گرفتن طبق فرمول بالا باید ضرب آنها جمع و ذخیره شود. مثلا شکل زیر نمونه ای از کاری است که برای چند سیگنال دریافتی میکنیم:



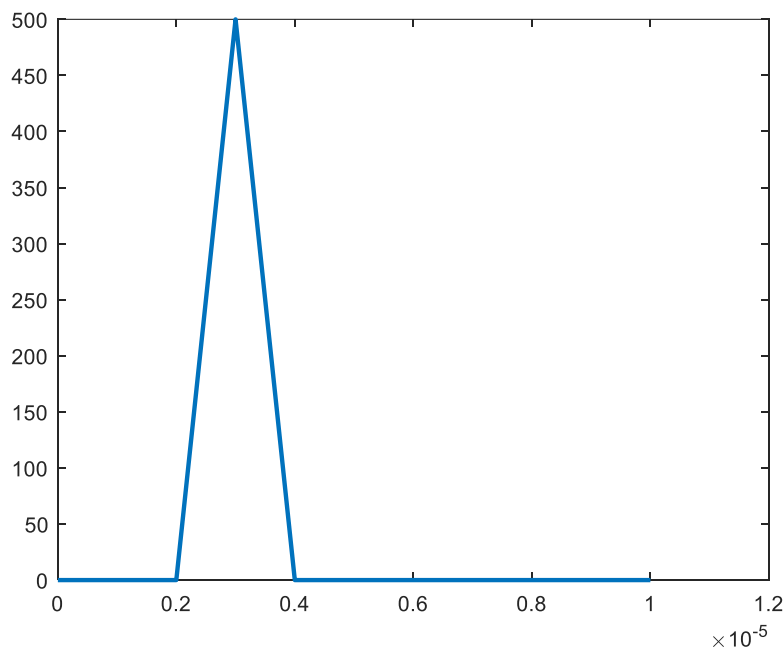
```

ts= 1e-9;
T= 1e-5;
tau= 1e-6;
t= 0:ts:T;
ro = p3_corfunc(t, tau, ts, Received_Signal);
plot(t,ro,'LineWidth',2);
[a, max_corr] = max(ro);
td = t(max_corr);
estimated_R = td * c / 2;
fprintf('Actual Distance: %.2f meters\n', R);
fprintf('Estimated Distance: %.2f meters\n', estimated_R);

```

به این ترتیب بدون نویز میتوان کورلیشن گرفت و تطابق را انجام داد.

Template matching is a technique that uses correlation to compare a reference signal (template) with a larger signal to determine the presence and location of the template. It is often used in image processing and pattern recognition applications.



Command Window

```

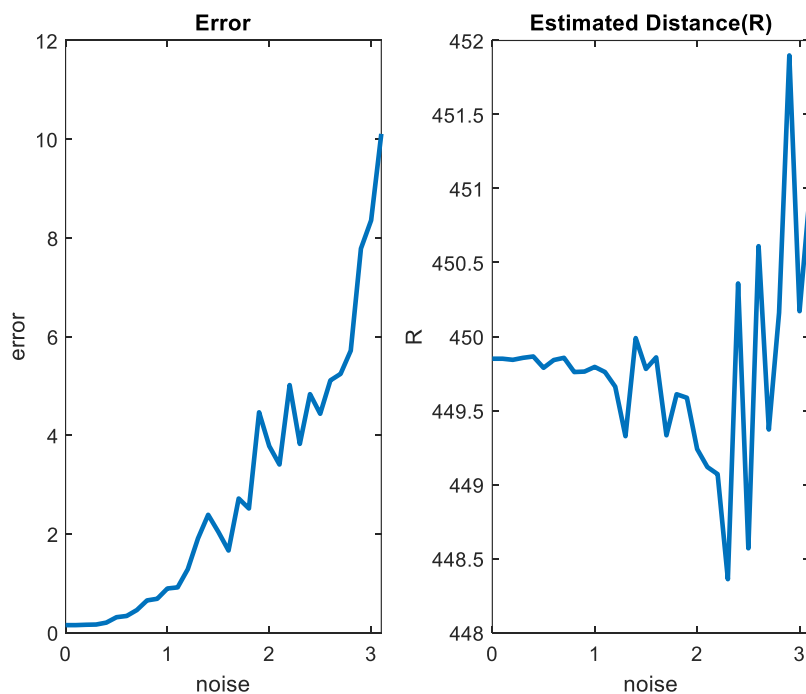
Actual Distance: 450.00 meters
Estimated Distance: 449.85 meters
Actual Distance: 450.00 meters
Estimated Distance: 449.85 meters

```

 >>

تمرین ۳-۴)

در این قسمت قصد داریم همان کار را با در نظر گرفتن نویز، یعنی به نحوی نزدیک شدن به حالت واقعی، انجام دهیم و سیگنال با نویزی تولید کنیم. برای این کار از توزیع گوسی استفاده میکنیم و دیتا میگیریم. برای نتیجه بهتر و نزدیک تر شدن میتوان این رندوم بودن را اصلاح کرد درواقع با تکرار این برداشت میتوان نتیجه نااریب تری گرفت. حالا با شروع نویز صفر و با اضافه شدن ۰.۱ در هر مرحله با همان الگوریتم پیش میرویم. بعلا اینکه درست تر خروجی بگیریم میزان اضافه شدن نویز ۰.۱ و تکرار آنچند مرتبه گرفته شده است و میانگین خطا را در نظر میگیریم. طبق آنچه که گفته شده است خطای فاصله یابی تا ۱۰ متر است یعنی باید حلقه بعد از آن متوقف شود (شرط توقف) بنابراین تا جایی نویز اضافه میکنیم که فاصله یعمی خطا به ۱۰ برسد.



```
Actual Distance: 450.00 meters  
Estimated Distance: 449.85 meters
```

```
fx >>
```



```

noise = 0;
k = 100;
R = 450;
noise_raise = 0.1;
errors = zeros(1,k);
Rs = zeros(1,k);
last_err = 0;
i = 1;
while(last_err < 10)
sum_err = 0;
sum_R = 0;
for j=1:100
ro = p3_corfunc(t, tau, ts, Received_Signal + noise * randn(size(Received_Signal)));
[a, max_corr] = max(ro);
estimated_R = t(max_corr) * c / 2;
sum_err = sum_err + abs(R - estimated_R);
sum_R = sum_R + estimated_R;
end
errors(i) = sum_err / 100;
Rs(i) = sum_R / 100;
last_err = errors(i);
i = i + 1;
noise = noise + noise_raise;
end
last = i - 1;
noise_signal= (0 : noise_raise : noise_raise * (last - 1));
figure;
subplot(1, 2, 1);
plot(noise_signal, errors(1:last), 'LineWidth', 2);
title('Error');
xlabel('noise');
ylabel('error');
subplot(1, 2, 2);
plot(noise_signal, Rs(1:last), 'LineWidth', 2);
title('Estimated Distance(R)');
xlabel('noise');
ylabel('R');

```